
CONSTRUCCIÓN DE SOFTWARE II

Análisis técnico de la Sesión 2 Prueba de concepto CacaoSelva

Estudiante: Juan Manuel Rengifo Fretel

Docente: Rios Rivera, Carlos Abraham

Semestre: 2026-II

Fecha: 26 de septiembre de 2026

Informe académico de definiciones, alcance y análisis técnico de los requerimientos de la sesión.

Resumen

Este informe analiza la Sesión 2, titulada *Prueba de concepto en Java con Arquitectura Limpia*, aplicada al caso CacaoSelva. El documento organiza el contenido en tres perspectivas: requerimientos funcionales, requisitos técnicos y arquitectura. El análisis identifica qué comportamiento debe ofrecer la solución, qué tecnologías y restricciones condicionan su implementación y cómo deben estructurarse las dependencias para proteger el dominio. La propuesta se orienta a una prueba de concepto académica compuesta por una API REST, una aplicación Desktop, un Monitor automático y, como actividad complementaria, un cliente web. La conclusión principal es que el éxito del ejercicio no depende únicamente de obtener respuestas correctas, sino de demostrar separación de responsabilidades, inversión de dependencias, manejo controlado de fallas y capacidad de evolución.

Palabras clave: CacaoSelva, Arquitectura Limpia, Java, requisitos, SOLID, API REST.

Índice

Resumen	1
1. Introducción	3
2. Alcance y criterios de análisis	3
3. Tabla 1. Requerimientos funcionales	4
3.1. Lectura técnica de los requerimientos funcionales	4
4. Tabla 2. Requisitos técnicos	4
4.1. Lectura técnica de los requisitos técnicos	4
5. Tabla 3. Arquitectura propuesta	7
5.1. Lectura técnica de la arquitectura	7
6. Trazabilidad y estrategia de validación	8
7. Conclusiones	8
Referencias	9

1 Introducción

La sesión propone construir una prueba de concepto, no un sistema productivo. Esta diferencia es importante: el alcance se concentra en demostrar que varias aplicaciones pueden consultar la misma información y que las reglas del negocio permanecen aisladas de las tecnologías externas. En consecuencia, el proyecto debe privilegiar claridad, trazabilidad y facilidad de explicación sobre características de producción como seguridad avanzada, alta disponibilidad o persistencia distribuida.

El caso utiliza tres lotes ficticios: Ana, con 120,5 kg y estado pendiente; Luis, con 80 kg y estado liquidado; y Rosa, con 95,25 kg y estado pendiente. A partir de ellos se obtiene un total de tres lotes, dos pendientes, uno liquidado y 295,75 kg. Estos datos constituyen la referencia funcional para validar la API, el Desktop y el Monitor.

La arquitectura descrita sigue el principio de que las capas externas pueden conocer a las internas, pero las internas no deben depender de las externas. Este criterio se relaciona con la independencia de frameworks y con la regla de dependencia de la Arquitectura Limpia (Martin, 2017).

2 Alcance y criterios de análisis

El análisis se organiza en tres tablas. La primera reúne los requerimientos funcionales y describe el comportamiento observable del sistema. La segunda agrupa los requisitos técnicos y explica las decisiones tecnológicas necesarias para materializar ese comportamiento. La tercera sintetiza la arquitectura propuesta, sus capas, contratos y responsabilidades.

Las tablas no representan código ni una implementación concreta. Funcionan como una especificación técnica para orientar el diseño, la construcción y la validación posterior mediante las pruebas P01 a P08. La distinción entre requisito funcional y técnico permite evitar que una tecnología se confunda con el resultado que el usuario necesita.

Ruta de lectura**Qué debe hacer** → **Con qué condiciones** → **Cómo se organiza**

3 Tabla 1. Requerimientos funcionales

3.1 Lectura técnica de los requerimientos funcionales

Los requerimientos RF01 a RF05 establecen el contrato mínimo de la API. Es especialmente relevante diferenciar los errores 400 y 404: el primero representa una solicitud inválida, mientras que el segundo representa una solicitud válida sobre un recurso inexistente. Esta distinción evita respuestas ambiguas y permite que los clientes reaccionen de manera correcta.

Los requerimientos RF06 a RF09 trasladan el contrato al Desktop. La interfaz no debe limitarse a presentar datos: también debe hacer visible el estado de la operación y comunicar la indisponibilidad de la API. El criterio más importante es que una falla de conexión no se presente como “cero lotes”, porque ambos estados tienen significados operativos diferentes.

Los requerimientos RF10 a RF15 definen el comportamiento del Monitor como observador periódico. Cada mensaje “Pendientes: 2” representa una observación del estado actual, no una acumulación de lotes. La recuperación ante fallas exige que el ciclo periódico sobreviva a una excepción temporal y vuelva a intentarlo.

4 Tabla 2. Requisitos técnicos

4.1 Lectura técnica de los requisitos técnicos

La combinación de Java y Maven permite organizar el proyecto como una solución multimódulo. Spring Boot y JavaFX cumplen funciones externas diferentes: la primera atiende solicitudes HTTP y la segunda presenta información al usuario. El Monitor, por su parte, se mantiene como un proceso Java independiente para demostrar que varios consumidores pueden usar el mismo contrato.

Cuadro 1: Requerimientos funcionales y análisis técnico

Código	Requerimiento	Análisis técnico
RF01	Obtener todos los lotes mediante <code>GET /lotes</code> .	La API debe retornar los tres registros disponibles en memoria.
RF02	Consultar un lote por identificador.	El sistema debe localizar un lote específico usando su ID.
RF03	Informar cuando un lote no existe.	Un ID válido pero inexistente debe producir HTTP 404 Not Found.
RF04	Rechazar identificadores con formato inválido.	Valores como <code>abc</code> no pueden convertirse en identificadores numéricos y deben producir HTTP 400 Bad Request.
RF05	Rechazar identificadores no permitidos.	Los valores menores o iguales a cero deben considerarse inválidos y producir HTTP 400.
RF06	Permitir consultar lotes desde Desktop.	La aplicación JavaFX debe disponer de una acción visible para iniciar la consulta.
RF07	Mostrar los lotes en una tabla.	La interfaz debe presentar ID, socio, peso y estado de cada lote.
RF08	Informar que una consulta está en proceso.	Mientras se realiza la petición, la interfaz debe mostrar un estado como “Consultando...”.
RF09	Informar cuando la API no está disponible.	Un error de comunicación debe mostrarse claramente y no confundirse con una lista vacía.
RF10	Consultar lotes automáticamente desde el Monitor.	El Monitor debe funcionar sin intervención del usuario.
RF11	Repetir la consulta cada 10 segundos.	Debe utilizarse una planificación periódica controlada.
RF12	Contar lotes pendientes.	El Monitor debe identificar los lotes en estado PENDIENTE y obtener el valor 2.
RF13	Registrar fallas de disponibilidad.	Una interrupción de la API debe registrarse sin provocar el cierre inesperado del Monitor.
RF14	Recuperarse después de una falla temporal.	El Monitor debe continuar ejecutándose y volver a consultar cuando la API se restablezca.
RF15	Detener el Monitor de manera controlada.	El proceso debe permitir finalizar sus recursos y tareas periódicas correctamente.
RF16	Consumir la API desde un cliente web.	El cliente web debe consultar el mismo endpoint <code>/lotes</code> .
RF17	Diferenciar error de conexión y lista vacía.	La aplicación debe distinguir entre una respuesta válida sin registros y una falla de comunicación.

Nota. Los códigos RF corresponden a la especificación funcional de la Sesión 2. La aceptación debe comprobarse mediante respuestas HTTP, observación de la interfaz y registros del Monitor.

Cuadro 2: Requisitos técnicos y análisis de implementación

Código	Requisito técnico	Análisis técnico
RT01	Utilizar Java.	Java proporciona el lenguaje base y las bibliotecas para los componentes.
RT02	Administrar la construcción con Maven.	Maven organiza módulos, dependencias, compilación y pruebas.
RT03	Implementar la API con Spring Boot.	Spring Boot proporciona el servidor HTTP, controladores REST y configuración de la API.
RT04	Implementar Desktop con JavaFX.	JavaFX permite construir la interfaz gráfica y sus controles visuales.
RT05	Implementar el Monitor como proceso Java independiente.	El Monitor puede iniciarse y detenerse separado de la API y Desktop.
RT06	Utilizar HTTP para la comunicación.	Los clientes se comunican con la API mediante solicitudes HTTP.
RT07	Utilizar JSON para el intercambio.	JSON transporta los lotes entre la API y los clientes.
RT08	Ejecutar la API en <code>http://localhost:5080</code>	La dirección y el puerto deben estar centralizados en la configuración.
RT09	Utilizar <code>java.net.http.HttpCli</code>	El adaptador HTTP encapsula el consumo mediante el cliente estándar de Java.
RT10	Ejecutar operaciones bloqueantes de forma asíncrona.	Las llamadas HTTP del Desktop no deben ejecutarse en el hilo de JavaFX.
RT11	Consultar aproximadamente cada 10 segundos.	La periodicidad debe implementarse con un planificador controlado.
RT12	Centralizar URL y tiempos de espera.	La configuración no debe repetirse en varios componentes.
RT13	Conectar tecnologías externas mediante adaptadores.	HTTP, JSON, JavaFX y almacenamiento deben mantenerse fuera del núcleo de negocio.
RT14	Utilizar Postman para comprobar la API.	Postman permite validar endpoints, respuestas y errores HTTP.
RT15	Registrar método, URL, código y cuerpo.	Las pruebas deben generar evidencia verificable del comportamiento de la API.

Nota. Los requisitos RT describen restricciones tecnológicas y de operación. No sustituyen los criterios funcionales: indican con qué condiciones debe alcanzarse el comportamiento esperado.

HTTP y JSON constituyen el límite de comunicación entre aplicaciones. Por ello, los detalles de construcción de solicitudes, conversión de respuestas y tratamiento de códigos

HTTP deben concentrarse en un adaptador. La interfaz gráfica y el Monitor solo deberían conocer una operación conceptual de consulta, no los detalles internos de la red.

El requisito RT10 es crítico para la experiencia de usuario. Una operación bloqueante ejecutada directamente en el hilo de JavaFX puede congelar la ventana. La solución debe realizar la consulta fuera de ese hilo y devolver el resultado a la interfaz de forma segura. Del mismo modo, RT11 exige planificación periódica en lugar de un ciclo infinito desorganizado.

5 Tabla 3. Arquitectura propuesta

5.1 Lectura técnica de la arquitectura

La arquitectura propuesta utiliza el dominio como centro estable. En él se ubican los conceptos que representan el negocio, mientras que la aplicación contiene los casos de uso que coordinan las operaciones. Los puertos expresan las necesidades de esas capas sin imponer una tecnología concreta.

La infraestructura implementa los contratos definidos por las capas internas. En esta prueba de concepto, el repositorio en memoria satisface el acceso a los lotes y el adaptador HTTP satisface la consulta realizada por los clientes. Si posteriormente se incorpora una base de datos, el cambio debe concentrarse en una nueva implementación del puerto, sin modificar las reglas de negocio.

La API, el Desktop y el Monitor son entradas diferentes al mismo problema. La API expone el contrato HTTP; el Desktop convierte el resultado en una experiencia visual; y el Monitor ejecuta observaciones periódicas. Esta separación aplica el principio de responsabilidad única y el principio de inversión de dependencias, dos prácticas centrales para reducir acoplamiento (Martin, 2008).

6 Trazabilidad y estrategia de validación

La trazabilidad relaciona cada grupo de requisitos con una forma de comprobación. RF01 a RF05 se validan mediante Postman y sus códigos HTTP. RF06 a RF09 se observan en la interfaz Desktop, incluyendo el estado de consulta y la respuesta ante la caída de la API. RF10 a RF15 se verifican mediante los registros del Monitor, su continuidad tras una falla y su detención voluntaria. RT01 a RT15 se inspeccionan en la configuración, los módulos, los adaptadores y la evidencia de las pruebas.

La validación completa debe incluir los escenarios P01 a P08 descritos en la sesión: listado, consulta individual, lote inexistente, identificadores inválidos, consulta desde Desktop, ejecución del Monitor, interrupción y recuperación de la API y detención controlada del Monitor. La evidencia recomendada incluye capturas de Postman, respuestas JSON, códigos HTTP, mensajes de la interfaz y registros de consola.

7 Conclusiones

La Sesión 2 define una prueba de concepto cuyo valor principal es arquitectónico. El sistema debe demostrar que una API, un Desktop y un Monitor pueden consultar la misma información sin duplicar reglas ni acoplar el núcleo a tecnologías externas.

Los requerimientos funcionales establecen el comportamiento observable; los requisitos técnicos fijan las herramientas y restricciones; y la arquitectura organiza las dependencias para proteger el dominio. Estas tres perspectivas son complementarias y deben revisarse conjuntamente.

La solución propuesta es apropiada para un contexto académico porque mantiene el alcance controlado: utiliza datos en memoria, operaciones de solo lectura y ejecución local. No pretende demostrar seguridad, persistencia real, rendimiento, alta disponibilidad ni preparación para producción. Su resultado esperado es una base comprensible y extensible que permita incorporar nuevas implementaciones sin alterar las reglas de negocio.

Referencias

Martin, R. C. (2008). *Clean code: A handbook of agile software craftsmanship*. Prentice Hall.

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Oracle. (s. f.). *Java documentation*. Recuperado el 26 de septiembre de 2026, de <https://docs.oracle.com/en/java/>

Spring. (s. f.). *Spring Boot reference documentation*. Recuperado el 26 de septiembre de 2026, de <https://docs.spring.io/spring-boot/index.html>

OpenJFX. (s. f.). *OpenJFX documentation*. Recuperado el 26 de septiembre de 2026, de <https://openjfx.io/>

Cuadro 3: Elementos de Arquitectura Limpia y responsabilidades

Elemento	Definición	Análisis técnico
Arquitectura Limpia	Modelo que protege las reglas de negocio frente a detalles tecnológicos.	Las dependencias deben dirigirse hacia las capas internas.
Dominio	Núcleo con las entidades y conceptos principales del negocio.	Debe ser independiente de Spring, JavaFX, HTTP, JSON y bases de datos.
Aplicación	Capa que contiene los casos de uso.	Define qué debe hacer el sistema sin conocer cómo se transportan o almacenan los datos.
Puertos	Interfaces que representan necesidades de la aplicación.	Permiten depender de abstracciones en lugar de implementaciones concretas.
Adaptadores	Componentes que conectan el núcleo con tecnologías externas.	Traducen HTTP, JSON, almacenamiento o interfaz gráfica sin contaminar el dominio.
API REST	Adaptador de entrada para solicitudes HTTP.	Recibe peticiones, invoca casos de uso y transforma resultados en respuestas HTTP.
Desktop	Interfaz gráfica para la interacción del usuario.	Debe delegar la consulta y no contener reglas de negocio ni detalles HTTP.
Monitor	Proceso automático que consulta y registra el estado.	Debe separar planificación, consulta, conteo y registro de errores.
Repositorio en memoria	Implementación temporal del acceso a lotes.	Permite demostrar la solución sin una base de datos.
Inversión de dependencias	Las capas internas dependen de abstracciones.	Permite reemplazar el almacenamiento con bajo impacto sobre los casos de uso.
Separación de responsabilidades	Cada componente posee una función claramente delimitada.	Controladores, casos de uso, repositorios, adaptadores y planificadores evolucionan de forma independiente.
Independencia del dominio	El dominio no conoce frameworks ni infraestructura.	Facilita pruebas, mantenimiento y sustitución de tecnologías.
Reutilización del núcleo	API, Desktop y Monitor comparten contratos y reglas.	Evita duplicar validaciones y lógica de negocio en cada aplicación.
Manejo de errores	Los errores se transforman en respuestas o mensajes controlados.	Se diferencian errores de entrada, recursos inexistentes y fallas de conexión.
Recuperación del Monitor	El proceso continúa después de una falla temporal.	Una interrupción de la API no debe detener permanentemente el Monitor.
Evidencia de validación	Las pruebas demuestran funcionamiento y	Deben comprobarse P01–P08, incluyendo fallas y detención controlada.